

The syllabus for Quiz-2 is Lectures 8 to 14: the λ -calculus (syntax, semantics, encodings), the simply typed λ -calculus, side effects, modular programming and monads. Unless a question says otherwise, you may use the encodings from the lectures by name without redefining them: *tru*, *fls*, *test*, *and*, *or*, *not*, *pair*, *fst*, *snd*, the Church numerals, *scc*, *plus*, *mult*, *is_zero*, *prd*, *sub* and the *Y* combinator. $\bar{n}$ denotes the Church numeral for n , and $\Omega = (\lambda x. x x) (\lambda x. x x)$. In OCaml questions, **let*** is the monadic bind from the lectures.

1. True or False. No explanation needed.

- (a) $\lambda x. x y z$ and $\lambda x. (x y) z$ are the same term.
- (b) $\lambda x. x y z$ and $\lambda x. x (y z)$ are the same term.
- (c) A closed λ -term (one with no free variables) must be in β -normal form.
- (d) A term in β -normal form is strongly normalising under every reduction strategy.
- (e) When $(\lambda x. x x) ((\lambda y. y) z)$ is reduced with the normal order strategy, the redex $(\lambda y. y) z$ gets reduced twice.
- (f) On the term in (e), call-by-value reaches the β -normal form in fewer steps than normal order.
- (g) Every term in β -normal form is also in $\beta\eta$ -normal form.
- (h) $\lambda x. f x =_{\beta} f$.
- (i) The term $\lambda x:A. x x$ can be typed in the simply typed λ -calculus if the type A is chosen suitably.
- (j) In the simply typed λ -calculus, if $\vdash M : A$ and $M \rightarrow M'$, then $\vdash M' : A$.
- (k) Since every well-typed term of the simply typed λ -calculus terminates, every untyped λ -term that terminates is well-typed.
- (l) **let** $x = \mathbf{ref} \ 0$ **in** **let** $y = \mathbf{ref} \ 0$ **in** $x == y$ evaluates to **true**.
- (m) **let** $x = \mathbf{ref} \ 0$ **in** **let** $y = \mathbf{ref} \ 0$ **in** $x = y$ evaluates to **true**.
- (n) Inside a structure, **open** M makes the definitions of M part of the structure's own interface.
- (o) Given **module type** $T = \mathbf{sig} \ \mathbf{type} \ t \ \mathbf{val} \ v : t \ \mathbf{end}$ and **module** $F (X : T) : T = X$, the types $F(M).t$ and $M.t$ are the same type for every module $M : T$.
- (p) Any type $'a \ t$ together with functions $\mathbf{return} : 'a \rightarrow 'a \ t$ and $\mathbf{bind} : 'a \ t \rightarrow ('a \rightarrow 'b \ t) \rightarrow 'b \ t$ is a monad.
- (q) In the option monad, **let*** $x = \mathbf{None}$ **in** e never evaluates e .

2. Syntax of the λ -calculus.

- (a) Make all the parentheses explicit in the following terms, using the scoping conventions discussed in class.
 - (i) $\lambda x. x \lambda y. y x z$
 - (ii) $(\lambda x. \lambda y. x) a \lambda z. z b$
 - (iii) $x \lambda x. x (\lambda y. y) x$
- (b) Circle the free variables in each of the following terms.
 - (i) $\lambda x. \lambda y. x z (\lambda z. z y)$
 - (ii) $(\lambda x. x y) (\lambda y. y x)$
 - (iii) $\lambda x. (\lambda x. x) x (\lambda y. x y z)$
- (c) Which of the following α -equivalences hold?
 - (i) $\lambda x. \lambda y. x y =_{\alpha} \lambda y. \lambda x. y x$
 - (ii) $\lambda x. x y =_{\alpha} \lambda y. y y$
 - (iii) $\lambda x. (\lambda x. x) x =_{\alpha} \lambda y. (\lambda x. x) y$
 - (iv) $\lambda x. \lambda y. x =_{\alpha} \lambda y. \lambda y. y$
 - (v) $\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x)) =_{\alpha} \lambda g. (\lambda x. g (x x)) (\lambda y. g (y y))$

(d) Using the OCaml type

```
type expr = Var of string | Lam of string * expr | App of expr * expr
```

from the lectures, write the abstract syntax tree of $(\lambda x. x y) z$.

3. Substitution. Recall the definition of capture-avoiding substitution from class:

$$\begin{aligned}
 x[s/x] &= s \\
 y[s/x] &= y && \text{if } x \neq y \\
 (\lambda x. t_1)[s/x] &= \lambda x. t_1 \\
 (\lambda y. t_1)[s/x] &= \lambda y. t_1[s/x] && \text{if } x \neq y \text{ and } y \notin FV(s) \\
 (\lambda y. t_1)[s/x] &= \lambda w. t_1[w/y][s/x] && \text{if } x \neq y \text{ and } y \in FV(s) \text{ and } w \text{ is fresh} \\
 (t_1 t_2)[s/x] &= (t_1[s/x]) (t_2[s/x])
 \end{aligned}$$

(a) Compute the following substitutions, showing each step.

- (i) $(\lambda x. y x)[(x x)/y]$
- (ii) $(\lambda y. x (\lambda x. x y))[z/x]$
- (iii) $((\lambda x. x y) (\lambda y. x y))[(\lambda z. z)/x]$

(b) Consider the following alternative rule for the case $(\lambda y. t)[s/x]$ where $x \neq y$ and $y \in FV(s)$. Instead of renaming y in t , rename y in s :

$$(\lambda y. t)[s/x] = \lambda y. t[(s[w/y])/x] \quad w \text{ fresh}$$

For example, under this rule $(\lambda y. x)[y/x] = \lambda y. w$. Is this a valid definition of substitution? If yes, justify briefly; if no, give a concrete counterexample.

4. Reduction strategies. Consider the terms

$$T_1 = (\lambda x. x x) ((\lambda y. y) z) \quad T_2 = (\lambda f. \lambda x. f (f x)) (\lambda y. y) a \quad T_3 = (\lambda x. y) \Omega$$

- (a) Reduce each of T_1, T_2, T_3 to β -normal form using the **normal order** strategy, showing every step and naming the rule ([N-APP], [N-CONG-1], [N-CONG-2], [N-ABS]) used at each step.
- (b) For each term, how many steps does **call-by-value** take to reach a β -normal form? (Or does it diverge?)
- (c) For T_1 and T_2 , what term does **call-by-name** stop at? Is it in β -normal form?
- (d) For each of the following terms say (1) whether it is weakly normalising and whether it is strongly normalising under full β -reduction, and (2) whether normal order, call-by-name and call-by-value reduction reach a β -normal form.

- (i) $(\lambda x. \lambda y. y) \Omega$
- (ii) $\lambda x. \Omega$
- (iii) $(\lambda x. x) (\lambda x. x x)$
- (iv) $(\lambda x. x x) (\lambda y. y)$

(e) True or false, for arbitrary terms M, M_1, M_2, N, N_1, N_2 :

- (i) If $M \rightarrow_{\beta^*} N_1$ and $M \rightarrow_{\beta^*} N_2$ and both N_1 and N_2 are in β -normal form, then $N_1 =_{\alpha} N_2$.
- (ii) If $M_1 =_{\beta} M_2$ and M_1 has a β -normal form, then M_2 has the same β -normal form (up to α).
- (iii) If $M_1 =_{\beta} M_2$ then $M_1 \rightarrow_{\beta^*} M_2$ or $M_2 \rightarrow_{\beta^*} M_1$.
- (iv) If $M \rightarrow_{\beta^*} N$ then $M =_{\beta} N$.

(f) Which of the following terms are in $\beta\eta$ -normal form? For those that are not, give the $\beta\eta$ -normal form.

- (i) $\lambda x. f x$ (ii) $\lambda x. f x x$ (iii) $\lambda x. x x$ (iv) $\lambda x. (\lambda y. y) x$ (v) $\lambda x. \lambda y. f x y$

5. λ -encodings.

- (a) Booleans. Without using *and*, *or* or *not* (you may use *tru* and *fls*), give encodings of
 - (i) *nand*, where *nand* $b\ c$ is *fls* exactly when both b and c are *tru*;
 - (ii) *xor*, where *xor* $b\ c$ is *tru* exactly when b and c differ.
- (b) Pairs. Give an encoding of *map_pair* such that $\text{map_pair } f \text{ (pair } u\ v) =_{\beta} \text{pair } (f\ u) (f\ v)$.
- (c) Church numerals.
 - (i) Which numeral is $\bar{2}\ \bar{3}$ β -equivalent to?
 - (ii) Which numeral is $\bar{3}\ \bar{2}$ β -equivalent to?
 - (iii) Which numeral is $\bar{2}\ scc\ \bar{5}$ β -equivalent to?
 - (iv) Give an encoding of *is_one*, which returns *tru* exactly on $\bar{1}$. You are not allowed to use the encoding of *equal*. However, you can use *is_zero*.
 - (v) Give encodings of *leq* (less than or equal) and of *max*.
 - (vi) Give an encoding of *square*.
 - (vii) Give a term F such that $Y\ F$ computes the Fibonacci function ($fib\ \bar{0} = \bar{0}$, $fib\ \bar{1} = \bar{1}$, $fib\ \bar{n} = fib\ \bar{n} - 1 + fib\ \bar{n} - 2$). You may use your *leq* from (v).

6. Simply typed λ -calculus. A, B, C are distinct base types.

- (a) Give typing derivation trees for the following judgements, naming the rule used at each step.
 - (i) $\vdash \lambda x:A. \lambda y:B. \langle y, x \rangle : A \rightarrow B \rightarrow B \times A$
 - (ii) $f:A \rightarrow B, p:A \times A \vdash f\ (\text{snd } p) : B$
 - (iii) $\vdash (\lambda p:1 \times 1. \text{fst } p)\ (\langle (), () \rangle) : 1$
- (b) For each of the following terms, give its type, or explain briefly why it has none.
 - (i) $\lambda p:A \times B. \langle \text{snd } p, \text{fst } p \rangle$
 - (ii) $\lambda f:A \rightarrow B. \lambda g:B \rightarrow C. \lambda x:A. g\ (f\ x)$
 - (iii) $\lambda x:A. x\ x$
 - (iv) $\text{fst } (\lambda x:A. x)$
 - (v) $\lambda x:A. \lambda y:A \rightarrow B. y\ (y\ x)$
- (c) Which of the following closed terms are *stuck*, i.e. not a value and with no reduction rule applicable?
 - (i) $\text{fst } ()$ (ii) $()\ ()$ (iii) $\text{fst } ((\lambda x:1. \langle x, x \rangle)\ ())$ (iv) $\lambda x:1. \text{fst } x$

7. Side effects.

- (a) For each program, state the value that **ans** is bound to after it executes.

(i) **let** $r = \text{ref } 1$
let $f\ () = (r := !r * 2; !r)$
let $a = f\ ()$
let $b = f\ ()$
let $\text{ans} = !r + a + b$

(ii) **type** $\text{cell} = \{ \text{mutable } v : \text{int} \}$
let $c1 = \{ v = 1 \}$
let $c2 = c1$
let $c3 = \{ v = 1 \}$
let $() = c2.v \leftarrow c2.v + 10$
let $\text{ans} = (c1.v, c3.v, c1 == c2, c1 == c3, c1 == c3)$

```
(iii) let make () = let c = ref 0 in (fun () -> c := !c + 1; !c)
      let f = make ()
      let g = make ()
      let _ = f ()
      let _ = f ()
      let x = f ()
      let y = g ()
      let ans = (x, y)
```

```
(iv) let ans =
      let x = ref 10 in
      let y = x in
      let z = ref !x in
      y := !y + 1;
      z := !z + 1;
      (!x, !z, x == y, x == z, x = z)
```

(b) Implement the following.

- (i) `swap : 'a ref -> 'a ref -> unit`, which exchanges the contents of the two cells.
- (ii) `make_counter : int -> (unit -> int)* (unit -> unit)`, where `make_counter init` returns a pair (`next`, `reset`): each call `next ()` increments a counter that starts at `init` and returns the new value; `reset ()` sets it back to `init`.
- (iii) `last_result : ('a -> 'b)-> ('a -> 'b)* (unit -> 'b option)`, where `last_result f` returns (`g`, `last`) such that `g` behaves like `f` and `last ()` returns `Some r` where `r` is the result of the most recent call of `g`, or `None` if `g` has not been called yet. For example, the following expression should evaluate to `Some 4`:

```
let (g, last) = last_result (fun x -> x + 1) in
let _ = g (g (g 1)) in
last ()
```

- (iv) Define the Fibonacci function *without* using the `rec` keyword, by “tying the knot” through a function reference as in the lectures.

(c) Using the doubly-linked list types from the lectures,

```
type 'a node = { v : 'a; mutable next : 'a node option; mutable prev : 'a
  node option }
type 'a dllist = 'a node option ref
```

implement

- (i) `last : 'a dllist -> 'a node option`, which returns the last node of the list, or `None` if it is empty;
- (ii) `duplicate : 'a dllist -> unit`, which duplicates every node of the list in place, keeping the next and prev links consistent. For example, if `l` is `1 <-> 2` then after `duplicate l`, `l` is `1 <-> 1 <-> 2 <-> 2`.

8. Modular programming. Consider

```
module type SET = sig
  type t
  val empty : t
  val add : int -> t -> t
  val mem : int -> t -> bool
  val size : t -> int
end

module ListSet : SET = struct
  type t = int list
```

```

let empty = []
let add x s = if List.mem x s then s else x :: s
let mem x s = List.mem x s
let size s = List.length s
end

```

- (a) For each expression, say whether it typechecks; if so, give its value, otherwise say why not.
- (i) `ListSet.size (ListSet.add 1 (ListSet.add 1 ListSet.empty))`
 - (ii) `ListSet.add 1 []`
 - (iii) `List.length ListSet.empty`
 - (iv) `ListSet.empty = []`
 - (v) `ListSet.mem 2 (ListSet.add 2 ListSet.empty)`
 - (vi) `let open ListSet in size empty`
- (b) Change one line so that `ListSet.empty = []` typechecks (without touching the **struct**). What does the module lose by this change? Give a concrete expression that becomes possible and breaks an invariant of the set.
- (c) Consider

```

module A = struct let x = 1 let y = 2 end
module B = struct include A let z = x + y end
module C = struct open A let z = x + y end

```

Which of the following names are bound, and to what? B.x, B.z, C.x, C.z, A.z.

- (d) Write a functor `MaxOf` that takes a module of type

```

module type ORDERED = sig type t val leq : t -> t -> bool end

```

and produces a module with a single function `max_list : O.t list -> O.t option` returning the largest element of the list (`None` for the empty list). Show how to instantiate it for `int` and for strings ordered by length.

- (e) Consider

```

module type T = sig type t val v : t end
module F (X : T) : T = struct type t = X.t * X.t let v = (X.v, X.v) end
module M = struct type t = int let v = 3 end
module N = F (M)

```

- (i) Does `fst N.v` typecheck? Why or why not?
- (ii) Change the first line of `F` so that `fst N.v + 1` typechecks and evaluates to 4.

9. Monads. Recall the option monad

```

let return v = Some v
let bind m f = match m with None -> None | Some v -> f v
let (let*) = bind

```

and the state monad `State (S : sig type t end)` from the lectures, with `get`, `put` and `run_state : 'a t -> state -> state * 'a`. In (d) and (e), `IntState = State (struct type t = int end)` has been opened.

- (a) Using `let*` and `return` (and no pattern matching on options), implement
- (i) `add_opts : int option -> int option -> int option`, which adds the two values if both are present and returns `None` otherwise;
 - (ii) `sum_opts : int option list -> int option`, which returns the sum of all the values if every element is a `Some`, and `None` otherwise.
- (b) What do the following evaluate to?

- (i) `bind (Some 3)(fun x -> if x > 2 then None else Some (x + 1))`
 (ii) `bind None (fun x -> Some (x + 1))`
- (c) Prove the following monad laws by unfolding the definitions:
 (i) right identity for the option monad: `bind m return = m` for every `m`;
 (ii) left identity for the state monad: `bind (return v) f = f v`.
- (d) What do the following evaluate to?
 (i) `run_state (let* s = get in let* _ = put (s + 1) in return s) 5`
 (ii) `run_state (let* _ = put 10 in let* s = get in return (s * 2)) 0`
 (iii) `run_state (let* a = get in
 let* _ = put (a * a) in
 let* b = get in
 let* _ = put (b + a) in
 return (a + b)) 3`
- (e) In the state monad, define `next : int t`, a computation that returns the current state and increments it, and then `label : 'a list -> (int * 'a) list t`, which pairs every element of the list with a fresh number obtained from `next`, in order. For example, `run_state (label ["a"; "b"; "c"]) 0` should produce the result `(3, [(0, "a"); (1, "b"); (2, "c")])`.
- (f) (Writer monad.) Define a module `Writer` with **type** `'a t = string * 'a`, `return`, and `(let*)`, such that with `log name f x = ("Called " ^ name ^ " on " ^ string_of_int x ^ ";", f x)` the program

```
let id x =
  let* v1 = log "inc" inc x in
  let* v2 = log "dec" dec v1 in
  return v2
```

gives `id 5 = ("Called inc on 5;Called dec on 6;", 5)`.

- (g) With the `Writer` monad of (f), `log` does not work for recursive functions written in the monad, because it ignores the log produced by the call `f x`. Rewrite `log` with type `string -> (int -> string * 'a) -> int -> string * 'a` so that

```
let rec fib n =
  if n < 2 then return 1
  else
    let* v1 = log "fib" fib (n-1) in
    let* v2 = log "fib" fib (n-2) in
    return (v1 + v2)
```

gives `fib 3 = ("Called fib on 2;Called fib on 1;Called fib on 0;Called fib on 1;", 3)`.